

TEAM STRUCTURE & PAGE OWNERSHIP

Group: AI-G01 | Course: SWE 3513 - Artificial Intelligence | Assignment 1

Member Roles & Page Ownership:

Role: Data & UX Lead (Member 1)

Responsibilities: Schema Validation, Feature Vectorization, Data Report, Dashboard PDF (Pages 1-2, 5)

Key Files: `src/data_pipeline.py`, `artifacts/data_report.json`, `AI_A1_G01_UIUX.pdf`

Role: Regression Engineer (Member 2)

Responsibilities: NumPy Gradient Descent, Scaling, Loss Curve, Regression Metrics, Derivation Notes

Key Files: `src/regression.py`, `artifacts/regression_metrics.json`, `artifacts/regression_loss.png`

Role: Classification Engineer (Member 3)

Responsibilities: Stratified Split, Logistic Regression, Metrics, Confusion Matrix, Error Cost Analysis

Key Files: `src/classification.py`, `artifacts/classification_metrics.json`, `artifacts/confusion_mat`

Role: Clustering & QA Engineer (Member 4)

Responsibilities: Standardization, k Comparison, Silhouette, Cluster Outputs, Cross-Pipeline QA

Key Files: `src/clustering.py`, `artifacts/clustering_metrics.json`, `artifacts/clusters.csv`, `artifacts`

Role: Reproducibility & Release Lead (Member 5)

Responsibilities: CLI Contract, Prediction Command, Requirements, README, Evidence, Final Package

Key Files: `run_all.py`, `predict.py`, `requirements.txt`, `README.md`, `evidence/`

Commit Requirements: Each member ≥ 2 meaningful commits to their assigned modules

Verification: Each member must explain/modify their section during live verification

SYSTEM OVERVIEW & OPERATIONAL DECISIONS

Stakeholder: Musanze HarvestLink Cooperative Operations Team

Three Operational Decisions:

Decision 1: Estimate Expected Harvest Weight

Task Type: Regression | Model: Linear Regression (NumPy GD)

Target: actual_yield_kg (kg) | Key Metrics: MAE=50.96, RMSE=59.15, R2=0.995

Decision 2: Flag Consignments Needing Dispatch Attention

Task Type: Binary Classification | Model: Logistic Regression

Target: dispatch_attention (0/1) | Key Metrics: Accuracy=1.0, F1=1.0, Prob threshold=0.5

Decision 3: Group Collection Points with Similar Profiles

Task Type: Unsupervised Clustering | Model: K-Means (k=3)

Target: 6 input features | Key Metrics: Silhouette=0.58, Inertia=156.7

System Flow:

CSV Upload (schema validated) -> Data Quality Report (SHA-256, stats, missing/duplicates)

-> Feature Vectorization (6 features, standardized) -> Model Inference (3 models)

-> Results: Yield prediction (kg) + Dispatch flag (0/1 + prob) + Cluster label (0/1/2)

-> Human Decision: Operations team reviews, applies domain knowledge, confirms dispatch

Input Schema: 9 columns (record_id + 6 features + 2 targets), 30 rows, no missing values, no duplicates

Dataset SHA-256: 70480135477a038edb41ec136b249f606176e616acf8c64a523773133f52c762

SCHEMA VALIDATION SPECIFICATION

Implementation: src/data_pipeline.py -> load_and_validate()

Required Columns (exact names, exact order not enforced):

record_id	text	Unique row identifier	Never used as model feature
plot_area_ha	numeric	Farm area (hectares)	Must be numeric
rainfall_mm	numeric	Recent rainfall estimate	Must be numeric
soil_ph	numeric	Soil acidity measure	Must be numeric
seed_kg	numeric	Seed quantity (kg)	Must be numeric
distance_km	numeric	Distance to collection point	Must be numeric
arrival_hour	numeric	Planned arrival hour (24h)	Must be numeric
actual_yield_kg	numeric target	Regression target	Must be numeric
dispatch_attention	binary target	Classification target	Must be 0 or 1 only

Validation Rules:

- All 9 columns present; missing columns raise ValueError with column names
- No unexpected/extra columns allowed
- Features + regression target must be numeric dtype
- dispatch_attention must contain only {0, 1}
- Missing values reported per column (count)
- Duplicate rows checked on features + targets (excluding record_id)
- SHA-256 computed on raw CSV file bytes for audit trail

Output: data_report.json containing:

- Row count, Feature count (6)
- Missing values per column
- Duplicate row count
- Descriptive statistics (mean, std, min, max, median per column)
- Group code (AI-G01)
- Dataset SHA-256 fingerprint
- Random seed (42)

FEATURE VECTORIZATION STRATEGY

Feature Matrix Construction:

1. Load CSV with pandas, validate schema
2. Separate identifier column (record_id) from features and targets
3. Feature columns (6): plot_area_ha, rainfall_mm, soil_ph, seed_kg, distance_km, arrival_hour
4. Convert to NumPy array: $X = df[FEATURE_COLS].values.astype(float) \rightarrow \text{shape} (n_samples, 6)$
5. Targets: $y_reg = df[actual_yield_kg].values.astype(float)$, $y_clf = df[dispatch_attention].values.astype(int)$
6. Train/test split: 80/20 stratified for classification, random for regression (seed=42)

Standardization (Supervised Models):

Scaler fitted ONLY on training data: $mean = X_train.mean(axis=0)$, $std = X_train.std(axis=0)$
 $std[std == 0] = 1.0$ to avoid division by zero
Transform: $X_scaled = (X - mean) / std$
Same scaler applied to test data and prediction inputs
Parameters saved in model artifacts for reproducibility

Clustering Standardization:

StandardScaler fitted on ALL data (unsupervised, no train/test split)
Targets (actual_yield_kg, dispatch_attention) EXCLUDED from clustering features
Scaler params saved in clustering_model.npz for prediction

No Hardcoded Values:

All dimensions derived from data shape at runtime
Column names from FEATURE_COLS constant
Works on hidden test data with same schema

MATHEMATICAL DERIVATION NOTES

Linear Regression with Batch Gradient Descent (src/regression.py)

Model: $y = Xw + b \rightarrow$ with bias trick: $y = X_{\text{bias}} @ w_{\text{bias}}$
where $X_{\text{bias}} = [1, X]$ shape $(n, d+1)$, $w_{\text{bias}} = [b, w]$ shape $(d+1)$

Loss Function (MSE):

$$L(w) = (1/n) * ||y - Xw||^2 = (1/n) * \sum((y_i - x_i^T w)^2)$$

Gradient:

$$dL/dw = (-2/n) * X^T @ (y - Xw) = (-2/n) * X^T @ \text{residuals}$$

Batch Gradient Descent Update:

$$w \leftarrow w - \alpha * dL/dw$$

$\alpha = \text{learning_rate}$ (0.01), epochs = 2000, tolerance = $1e-6$

Algorithm:

1. Add bias column: $X_{\text{bias}} = \text{hstack}([\text{ones}(n,1), X_{\text{scaled}}])$
2. Initialize weights: $w \sim N(0, 0.01^2)$ with seed=42
3. For each epoch: $y_{\text{pred}} = X_{\text{bias}} @ w$; $\text{loss} = \text{MSE}(y, y_{\text{pred}})$;
 $\text{grad} = (-2/n) * X_{\text{bias}}.T @ (y - y_{\text{pred}})$; $w \leftarrow w - \text{lr} * \text{grad}$
4. Track train_loss and val_loss per epoch
5. Early stop if $|\text{prev_loss} - \text{curr_loss}| < \text{tolerance}$

Metrics:

$$\text{MAE} = \text{mean}(|y - y_{\text{pred}}|)$$

$$\text{RMSE} = \sqrt{\text{mean}((y - y_{\text{pred}})^2)}$$

$$R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}} \text{ where } \text{SS}_{\text{res}} = \sum((y - y_{\text{pred}})^2), \text{SS}_{\text{tot}} = \sum((y - \text{mean}(y))^2)$$

Classification (Logistic Regression):

$$P(y=1|x) = 1 / (1 + \exp(-(w^T x + b)))$$

Trained with sklearn LogisticRegression(max_iter=1000, random_state=42)

Cost Analysis: False Negative (miss dispatch) > False Positive (extra attention)

Clustering (K-Means):

Minimize: $\sum_{i=1}^n ||x_i - c_{\{z_i\}}||^2$ where $z_i = \text{argmin}_k ||x_i - c_k||^2$

k evaluated 2..5 via silhouette_score; best k=3 (sil=0.58)

CORE PIPELINE IMPLEMENTATION

Entry Point: run_all.py

```
def main():  
    1. Parse CLI args: --data, --output, --group  
    2. Load & validate CSV -> DataFrame (30, 9)  
    3. Generate data_report.json (stats, SHA-256)  
    4. Separate features/targets: X (30,6), y_reg (30,), y_clf (30,)  
    5. Train regression: train_regression(X, y_reg) -> weights, scaler, metrics, loss_history  
    6. Save regression artifacts: metrics.json, loss.png, model.npz  
    7. Train classification: train_classification(X, y_clf) -> model, scaler, metrics, CM  
    8. Save classification artifacts: metrics.json, cm.png, model.joblib  
    9. Train clustering: train_clustering(X) -> labels, centers, scaler, k_eval  
    10. Save clustering artifacts: metrics.json, clusters.csv, plot.png, model.npz  
    11. Print summary metrics to stdout
```

Key Implementation Details:

Fixed SEED=42 used in: np.random.seed, train_test_split, KMeans, LogisticRegression, PCA
Scalers fitted on training data only (supervised); all data for clustering
Regression: gradient_descent() returns (weights, train_losses, val_losses)
Classification: stratified split preserves class distribution
Clustering: silhouette_score for k=2..5, automatic best-k selection
All artifacts saved to --output directory; models to models/
SHA-256 computed via hashlib.sha256 on raw file bytes

Prediction CLI: predict.py

Accepts --record JSON with 6 feature fields
Validates: all fields present, all numeric types
Loads all 3 models from models/ directory
Returns JSON: regression_prediction_kg, classification_prediction,
 classification_probability, cluster_label, group_code, model_version
Rejects invalid input with clear error message

UNIT TESTING SUIT

Test Strategy: Each module has if `__name__ == "__main__"` block for isolated testing

Integration: Full pipeline test via `run_all.py` in clean venv

Module-Level Tests:

data_pipeline.py

- Load CSV -> validate schema (30, 9)
- Check missing values -> all zeros
- Check duplicates -> 0
- Separate features -> `X(30,6)`, `y_reg(30,)`, `y_clf(30,)`
- Generate `data_report.json` -> valid JSON with all required keys

regression.py

- Train on `X`, `y_reg` -> returns `weights(7)`, `metrics`, `loss_history`
- Metrics: MAE, RMSE, R2 computed on test set
- Loss decreases monotonically (verified)
- Save/load model -> weights match
- Predict on new sample -> returns scalar
- Cross-check: vs `sklearn.LinearRegression` on same split

classification.py

- Stratified split -> class distribution preserved
- Train `LogisticRegression` -> converged (`max_iter=1000`)
- Metrics: Acc, Prec, Rec, F1, Confusion Matrix
- Error cost analysis: FN > FP documented
- Save/load model -> predictions match

clustering.py

- Evaluate `k=2,3,4,5` -> silhouette scores computed
- Best `k` selected automatically (max silhouette)
- KMeans fitted -> labels for all 30 samples
- PCA plot generated -> 2D projection
- Save/load model -> cluster assignment matches

Integration Tests (from TEST_LOG.md):

- Clean venv setup + `pip install -r requirements.txt`
- Full pipeline run -> all 8 artifacts + 3 models generated
- Regression metrics match expected ranges
- Classification metrics: Acc=1.0, F1=1.0
- Clustering: `k=3`, silhouette=0.58
- Prediction valid input -> returns all 6 output fields
- Prediction missing fields -> JSON error with field names
- Prediction wrong type -> JSON error with field name
- Hidden data simulation -> pipeline runs without code changes
- Reproducibility: 3 runs -> identical metrics, identical SHA-256

DECISION DASHBOARD TERMINAL INTERFACE

CLI Commands:

Full Pipeline:

```
python run_all.py --data data/AI_A1_G01.csv --output artifacts/ --group AI-G01
```

Prediction:

```
python predict.py --record '{"plot_area_ha":1.2,"rainfall_mm":81,...}'
```

Validated via Python -c to avoid shell escaping issues

Pipeline Output (stdout):

```
Loading data from data\AI_A1_G01.csv...
Data shape: (30, 9)
Generating data report...
Data report saved. SHA-256: 70480135477a038edb41ec136b249f606176e616acf8c64a523773133f52c762
Training regression model...
Regression metrics: MAE=50.96, RMSE=59.15, R2=0.9949
Training classification model...
Classification metrics: Acc=1.0000, F1=1.0000
Error cost: False Negative more costly than False Positive...
Training clustering model...
Clustering: k=3, Silhouette=0.5801
Interpretation: Clusters are statistical groupings...
Pipeline complete! Artifacts saved to: artifacts
{"regression_prediction_kg": 2684.4,
 "classification_prediction": 0,
 "classification_probability": 0.46,
 "cluster_label": 1,
 "model_version": "1.0.0"}
```

Prediction Output (JSON):

Validation Error Examples:

```
{"error": "Missing required fields: ['soil_ph', 'seed_kg', 'distance_km', 'arrival_hour']"}
{"error": "Field 'plot_area_ha' must be numeric, got str"}
{"error": "Invalid JSON: Expecting property name..."}
```


VERSIONED COMMIT HISTORY

Git Repository: [GitHub URL]

Final Commit Hash: [to be filled at submission]

Required: Each member ≥ 2 meaningful commits to their modules

Expected Commit Pattern:

Member 1 (Data & UX):

- feat(data): add schema validation and SHA-256 computation
- feat(data): implement data_report.json generation
- docs(uiux): add dashboard design PDF pages 1-2, 5

Member 2 (Regression):

- feat(reg): implement NumPy batch gradient descent
- feat(reg): add feature standardization and loss tracking
- feat(reg): implement MAE/RMSE/R2 metrics

Member 3 (Classification):

- feat(clf): add stratified split and LogisticRegression
- feat(clf): implement confusion matrix and metrics
- docs(clf): add error cost analysis (FN > FP)

Member 4 (Clustering & QA):

- feat(cluster): implement k=2..5 evaluation with silhouette
- feat(cluster): add PCA visualization and cluster CSV
- test(qa): cross-pipeline validation and TEST_LOG.md

Member 5 (Reproducibility):

- feat(cli): implement run_all.py and predict.py contracts
- feat(cli): add input validation and model loading
- docs: requirements.txt, README.md, evidence files
- chore: final package assembly and submission prep

Verification Requirements:

GitHub commit hash in Moodle must match submitted ZIP

Dataset must not be renamed, replaced, extended, or modified

SHA-256 in Moodle must match run_all.py output

README.md must contain: setup/execution commands, Python version, members, roles, repo URL, commit hash, evidence files

AI_A1_G01_CONTRIBUTIONS.pdf must be signed by every member

AI_USE.md must disclose every generative AI tool used, purpose, prompts, affected files, verification

All artifacts must be generated by submitted code (not manual)